MediAgent AI team learning guide

A plain language handbook for Techtonics

For Apsara, Vaibhav Sharma and Kajal Rajput

Project version reviewed on 30 October 2026

MediAgent AI combines local AI classification with a patient dashboard and saved visit reports. A second workflow retrieves medical reference material from MedlinePlus and asks the local model to summarize it. This guide explains the working code so every teammate can describe the design, operate the demo, and answer technical questions.

The main idea is simple: Python manages the workflow, Ollama runs a pretrained language model, and SQLite remembers the encounters. A clinician still needs to interpret the output. The prototype has not established clinical triage accuracy.

How to use this guide

Read pages 2 to 9 to understand the product. Read pages 10 to 14 for code, operation, testing and limitations. Use pages 15 and 16 for rehearsal. Pages 17 and 18 help you maintain the project and learn unfamiliar terms.

2 The parts of the application
3 What the AI does
4 One triage submission from start to finish
5 Structured output and validation
6 The dashboard and its statuses
7 Patient identity and returning visits
8 Storage and data retention
9 Medical knowledge search
10 A guided tour of the code
11 Starting and operating the app
12 Speed and troubleshooting
13 Software tests and clinical evidence
14 Privacy and accurate project claims
15 A demonstration your team can rehearse
16 Questions judges may ask
17 Maintaining and extending the project
18 Glossary and further reading

Learning goal: explain where the AI is used, where ordinary Python rules are used, what survives a restart, and what the system cannot reliably decide.

2 The parts of the application

Think of the app as a small reception desk with an assistant and a filing cabinet. The screen collects information. The assistant suggests a provisional classification. The filing cabinet keeps the record. Each part has a different job.

Part	Job in this project
Browser	Displays the interface and sends clicks and form input to the app.
Python and Streamlit	Build the form, run the workflow, show warnings, search records and display reports.
Ollama	Runs the selected model on the laptop and accepts requests from Python.
Qwen2.5 3B	The default pretrained model, selected as qwen2.5:3b. It generates classifications and summaries.
SQLite	Stores patient IDs, encounters, model output and care status in a local file.
MedlinePlus	Supplies external health-topic results for the reference-search workflow.

Where these parts run

The browser usually opens http://localhost:8501. The Streamlit Python process runs the interface and application logic. Ollama is a separate local service at http://localhost:11434. The app sends classification requests to /api/chat. SQLite is a file that Python opens directly, rather than a separate database server.

Localhost means this same computer. The numbers 8501 and 11434 are ports, like different doors for two services. A public website cannot reach your laptop through its own localhost address.

Two workflows share the model connection

ER triage sends the current age, gender and clinical description to Ollama, then saves the result. Knowledge Search first retrieves external source snippets, then sends the question and those snippets to Ollama. Triage does not retrieve guidelines or inspect MedlinePlus before classifying.

Both the interface and most server-side logic live in app.py. There is no separate React website or FastAPI service in this implementation. Streamlit handles the browser interface and Python application execution together.

3 What the AI does

A pretrained model supplies language understanding

A language model generates text using patterns learned during earlier training. Our team downloads an existing model. We have not trained a medical model, fine-tuned it, or taught it new medical facts by saving patient records. Inference means using that existing model to produce an answer.

Ollama is the runner, while qwen2.5:3b is the default model being run. The model name includes 3b, meaning roughly three billion model parameters. Parameters are learned numerical values inside the model, not three billion patient records or a measure of clinical accuracy.

Instructions guide each request

A prompt is the text sent to the model. Our system prompt explains the classification task and constraints. A separate user message contains the patient fields as JSON. The prompt asks the model to consider the full description, distinguish negation and history, explain uncertainty, and avoid inventing findings.

A current symptom, a symptom denied by the patient, and a symptom that occurred years ago have different meanings. Semantic classification tries to interpret those relationships rather than requiring an exact phrase from a Python keyword list. It can still misunderstand them.

What remains ordinary programming

Python checks the response format, sorts priority labels, saves records, matches names in search, and chooses which warning banner to display. The model does not decide which database row to update or automatically mark a patient as attended.

Some text rules still exist for reference-query preparation and cleaning markup. Removing fixed safety keywords did not remove all text processing from the app.

What this architecture is called

It is a local language-model application with a structured classification workflow. The project name contains Agent, but the app does not run an autonomous planner that chooses tools or repeatedly takes actions. Python controls a fixed sequence.

The code uses no LangChain, vector database, embeddings pipeline or model-training pipeline. SQLite supplies application memory for report retrieval. It does not change the model. The project uses a general pretrained model, not a model that this team has established as clinically reliable.

4 One triage submission from start to finish

Follow this sequence while looking at Add a patient / run triage. It connects the visible interface to the actual functions.

1. Register the encounter. Choose New patient or Returning patient. A returning visit requires selecting an existing patient. Enter the current age, gender and clinical description. Name and reference note are optional.

2. Submit the form. Streamlit makes the values available together. Empty clinical text produces a warning and no classification. An unavailable database disables submission so the app does not intentionally create an unsaveable record.

3. Save a Pending row. create_encounter allocates or reuses the patient ID, creates an internal encounter key, and stores the input. Care starts as Waiting. Saving first helps retain evidence of a failed or interrupted attempt.

4. Build the request. classify_patient prepares system instructions, the required output schema and the current patient fields. The dedicated name and reference fields are not included. Previous reports are not included automatically.

5. Call Ollama. call_ollama posts to the local chat endpoint. It waits for a complete response because stream is false. Temperature is zero for classification, and generation is limited to 512 tokens.

6. Validate the answer. The parser checks fields, types, allowed priorities and one emergency consistency rule. Invalid output gets one correction request to the model. Network failure is not retried through this correction loop.

7. Save the outcome. A valid result changes classification status to Complete and saves the JSON. A failed request changes it to Failed and stores an error when saving remains possible.

8. Refresh the screen. The dashboard reads the database, groups encounters, shows relevant warnings and opens the submitted report. The classification itself supplies the report, so there is no second prose-summary call.

Important consequences

Every Run Triage Analysis submission creates a new encounter. It does not update an earlier report in place. Generating a report also does not change care status to Completed. A person uses the care controls separately.

If generation succeeds but saving fails, the app keeps the result in the current session and warns that it is unsaved. That temporary copy can disappear when its session is lost and is not a database backup.

5 Structured output and validation

JSON is a text format with labelled fields. It lets Python read a result reliably instead of guessing which paragraph contains the priority. A schema describes the fields and the types of values allowed.

This simplified example illustrates the response shape. It is not a prediction for a real patient.

```
{
  "priority": "Unable to determine",
  "immediate_emergency": false,
  "concerns": ["Human assessment and clarification needed"],
  "historical_context": [],
  "reasoning": "The description lacks enough context.",
  "uncertainty": "Current circumstances are unknown."
}
```

The schema requires exactly these six fields. priority must be Critical, Urgent, Non-urgent or Unable to determine. immediate_emergency must be a JSON boolean, true or false. concerns and historical_context must be lists of nonempty strings. reasoning and uncertainty must be nonempty strings.

What the parser checks

_parse_classification reads the JSON. It rejects missing or extra fields, incorrect value types and unsupported labels. If immediate_emergency is true, priority must be Critical. The parser asks the model to correct an inconsistent response rather than inventing a new priority itself.

Some local outputs incorrectly escape apostrophes. The parser attempts only that narrow syntax repair and then validates again. It does not accept arbitrary prose or a string such as "false" where a boolean is required.

Why correction can take extra time

classify_patient allows at most two generation calls: the first answer and one correction. The correction includes the rejected answer and the validation issue. If it still fails, the app reports that no usable classification was produced. There is no hidden keyword-classification fallback.

What validation does not prove

Valid JSON can contain a medically incorrect conclusion. The field called reasoning is a generated explanation, not a verified record of the model's internal decision process. The emergency flag being false does not prove safety.

6 The dashboard and its statuses

The dashboard shows encounters. A person with two visits can appear twice, under the same patient ID. Matching patients currently counts matching encounter rows, not distinct people.

Three separate concepts

Concept	Possible values	Meaning
Model priority	Critical, Urgent, Non-urgent, Unable to determine	A provisional suggestion from the model.
Classification status	Pending, Complete, Failed	Whether the app obtained and saved a usable response.
Care status	Waiting, In care, Completed	A manual record of attendance for that encounter.

Complete means a report exists. Completed means a person marked care completed. Neither proves recovery, discharge readiness or clinical safety.

How ordering works

Active encounters with completed classifications appear as Critical first, then Urgent, then Non-urgent. Within a priority, an immediate-emergency flag comes first, followed by earlier arrival. An internal key breaks any remaining tie.

Unable to determine, Pending and Failed appear in Needs assessment. This is a separate group, not a queue of people who can safely wait. Completed-care records appear in a retained-records section instead of the active queue.

How warnings appear

render_review_warning shows an emergency message when the emergency flag is true. Critical or Urgent produces a prominent assessment warning. Unable to determine produces a clarification warning even when the concerns list is empty. A Non-urgent result with listed concerns also produces a review warning.

Python controls the banner wording, while Ollama supplies the classification. Warnings depend on the model result and appear after it returns. There is no independent instant keyword safety screen in this version.

Care actions

Start care changes Waiting to In care. Mark care completed records Completed and a completion time. Reopen encounter returns it to Waiting and clears the current completion time. Inputs and the model report remain saved. The database does not keep a full audit trail of every status change.

7 Patient identity and returning visits

One public patient ID

The interface shows a numeric patient ID starting at 10001. SQLite allocates the next number. The patient keeps it across explicitly linked returning visits. It is an identifier, not a security token or a diagnosis. As the database grows, the sequence can eventually require more than five digits.

The database also needs an internal key for each encounter so two reports do not overwrite each other. Keys such as V1 remain implementation details and are not displayed as a second public ID. Older databases may retain long legacy internal keys to preserve records.

A concrete example

Register the fictional patient Demo Asha as New patient. The app might assign 10001 and save the first report. Later, choose Returning patient and select 10001 before submitting current information. The second report belongs to 10001 but has its own arrival time, inputs, output and care status.

Visit history lists these encounters by arrival time and care status. Clicking a date loads that report from SQLite. It does not call Ollama or rewrite earlier results.

Why names do not establish identity

Different people can have the same name, and one person can use different spellings. The app requires explicit selection of an existing patient for a return. Choosing New patient again creates another identity, even if the name matches.

Names are stored on encounter rows. Editing a name in one report changes that encounter only, not every earlier visit. The returning-patient selector uses the latest saved encounter for each ID to display a name. There is no central editable demographic profile yet.

Registration details to check

Returning-patient selection preloads the saved name. Age, gender and symptoms remain current form fields, not a complete automatic copy of the old visit. Confirm them before submission. History is available to the user but is not silently fed into the classification request.

Upgrading earlier records

The storage code adds columns and upgrades older IDs without deleting reports. Legacy encounters without an established patient link receive separate identities. The app does not merge old records because their names look similar.

8 Storage and data retention

The default database is C:\mediagent_medical_search\data\triage.sqlite3. SQLite stores records in this local file. Python normally includes the sqlite3 interface used here, so no MongoDB service is required for the prototype.

Database table	What it contains
patients	One numeric ID per identity. It currently holds IDs only, not a full patient profile.
encounters	One row per visit, including patient link, input, classification and care status.
sequences	A counter for internal encounter keys.
sqlite_sequence	SQLite bookkeeping for automatically allocated numeric patient IDs.

What one encounter remembers

Identity and timing include internal id, patient_id, patient_name, reference, created_at, visit_type and an internal visit_code. The patient ID links deliberately connected visits.

Clinical input includes age, gender and symptoms. model records the selected model name. Classification fields include status, the JSON classification and error. Care fields are care_status and completed_at. Timestamps are stored in UTC and shown in IST.

The classification JSON contains priority, emergency flag, concerns, historical context, reasoning and uncertainty. The report is assembled from those saved fields. No separate patient PDF is automatically generated.

What survives closing the browser

Database records survive refreshes and normal app restarts. Streamlit session state mainly remembers the selected encounter, temporary messages and any generated-but-unsaved result. This is temporary UI memory, not permanent storage. Reference answers and sources also live in session state rather than patient tables.

Completing care updates fields instead of deleting a row. Opening a saved report reads SQLite without running the model. A name or ID search reads stored records, then filters them in Python with case-insensitive partial matching.

Database operation and backup

_connect opens a short-lived connection, creates or upgrades required structures, and closes it afterward. Changes commit together when an operation succeeds. MEDIAGENT_DB_PATH lets tests or demo runs select a different file.

For a simple backup, stop the app and copy the database file to a safe location. The data folder is excluded from source control, but that does not encrypt or back it up. Deleting the file removes the saved application data.

9 Medical knowledge search

This tab answers an educational reference question using retrieved snippets. It is separate from triage and does not create a patient record.

Retrieval before generation

1. Prepare a search. _medlineplus_query_variants extracts English words, removes common question words, tries a few remaining keywords together, and then tries individual terms when needed. This is a simple heuristic, not an AI search planner.

2. Request results. requests.get calls https://wsearch.nlm.nih.gov/ws/query with db=healthTopics. The UI requests up to two results. Each network request has an eight-second timeout.

3. Parse XML. _parse_medlineplus_xml extracts title, URL and summary or snippet. _clean_medlineplus_text decodes HTML entities, strips markup and normalizes spacing. XML is the structured text format the service returns.

4. Build context. The model receives the question and each source number, title, URL and up to 500 characters of retrieved summary. The app does not download and read every full linked page.

5. Summarize locally. The prompt asks Ollama to use only the supplied material, acknowledge gaps and cite source numbers such as [1]. It requests roughly 70 to 100 words and limits generation to 125 tokens. These settings do not guarantee a word count or a complete answer.

6. Show the answer and links. Users can open full sources. If retrieval fails or returns no usable sources, the app does not generate an unsupported fallback answer.

Caching and freshness

search_medlineplus uses st.cache_data with a one-hour lifetime. Repeating a matching search can reuse retrieved results. The cache covers retrieval, not a permanent record of every model answer. A cached result may be up to one hour old.

The scope of this search

The service searches MedlinePlus health topics. It is not PubMed literature search, a comprehensive drug-label database, or the separate MedlinePlus Connect service. Some drug or specialist questions will need another reference.

This retrieval-then-generation pattern is often called RAG. It uses no embeddings or vector database here. Source-only instructions help, but the code does not independently prove that every generated claim is supported by the cited snippet.

10 A guided tour of the code

The files that run the product

app.py is the entry point. It defines page appearance, both tabs, model settings, forms, dashboard rendering, care callbacks, reports and MedlinePlus functions. Streamlit executes the script again on many interactions. Conditions around submit buttons prevent those reruns from repeatedly generating reports.

safety_classifier.py defines allowed labels, CLASSIFICATION_SCHEMA, CLASSIFICATION_SYSTEM, _parse_classification and classify_patient. It prepares and validates classification separately from layout. It receives call_ollama as an argument, which also lets tests substitute a fake model function.

patient_store.py owns SQLite operations. create_encounter stores a visit. finish_encounter saves success or failure. get_encounter and list_encounters retrieve records. update_care_status and update_patient_name edit fields. filter_encounters searches records, and dashboard_groups sorts and groups them.

requirements.txt lists Streamlit and Requests. Modules such as sqlite3, json, re, datetime, html and xml.etree.ElementTree are part of Python's standard library in this environment.

A simplified trace

```
On Run Triage Analysis
    save input as Pending
    ask Ollama for structured classification
    validate and allow one correction if needed
    save Complete or Failed outcome
    read encounters and show queue and report

On report selection or care update
    read or update SQLite
    refresh the displayed records
    do not run the model again
```

Other files you will see

test_patient_dashboard.py checks storage and UI behavior. test_safety_classifier.py checks parsing and correction. test_triage_warnings.py checks visible warnings. test_medlineplus_parser.py tests an extracted helper in _medical_search_under_test.py. The hackathon_deck folder contains presentation artifacts, not runtime classification logic.

An import check in app.py reloads patient_store when specific required functions are absent from its cached module. It helps with one stale-module problem. It is not a universal hot-update system. Restarting Streamlit remains the reliable way to load all changes.

11 Starting and operating the app

Before the demonstration

Use C:\mediagent_medical_search. Python and the project packages must be available. Install Ollama using its official installer if needed, and keep it running. Download the model before attempting offline triage.

```
cd C:\mediagent_medical_search
python -m pip install -r requirements.txt
ollama pull qwen2.5:3b
python -m streamlit run app.py
```

The pull command needs internet and downloads the model. It is not required at every launch after installation. Open the URL printed by Streamlit, normally http://localhost:8501. Streamlit may choose another port if 8501 is busy.

A normal session

1. Keep the terminal and Ollama running. Test local model connection in the sidebar before presenting. That test calls the model and can take time.

2. Choose New patient for a different person. Choose Returning patient and confirm the numeric ID for another visit by an existing person. Every submit creates a separate encounter.

3. Open reports from patient rows or Visit history. Mark care status manually. Combine All patients, Active patients or Completed patients with name/ID search to locate records.

4. Refresh dashboard reads records added by other sessions. There is no automatic live-refresh timer in the current app.

Stopping and updating

Press Ctrl+C in the Streamlit terminal to stop the app. To load changed code, stop and run the same Streamlit command again. Closing the browser alone does not necessarily stop the Python server. Stopping the app does not erase SQLite data.

Useful checks

```
ollama list
python -m unittest discover -v
```

ollama list shows installed model names. The test command runs software checks with temporary test databases. Keep the exact model name in the sidebar, including :3b. Do not rename, move or delete the database during an active demo.

12 Speed and troubleshooting

Why generation takes time

The model loads weights when needed, processes the prompt, then generates tokens, which are small pieces of text. CPU/GPU availability, memory, input length, output length and simultaneous activity affect delay. A correction adds another generation call.

The app avoids a second summary call, limits triage to 512 tokens, requests concise fields and asks Ollama to keep the model loaded for ten minutes. stream=false means the page waits for the full response. keep_alive is a request to the runner, not a guarantee of instant responses.

Temperature zero reduces variability but does not ensure correctness or identical answers everywhere. The model timeout is 180 seconds per request, not a guaranteed overall wait. Correction can add another request.

Common problems

Cannot connect to Ollama: confirm it is running and use the connection test. Starting Streamlit does not automatically start Ollama.

Model not found: compare the sidebar name with ollama list. Download a missing model with ollama pull qwen2.5:3b. Changing the name field does not install it.

Invalid classification: output still failed the schema after correction. A Failed record is not a Non-urgent result. Review the failed encounter before creating repeated duplicate submissions.

ImportError after an update: restart Streamlit. Confirm that matching app.py and patient_store.py files are in the same folder. Cached-module recovery covers only specific missing functions.

Storage unavailable or an unsaved report: check disk space, write access and MEDIAGENT_DB_PATH. Keep an unsaved-report page open because its temporary result can disappear with the session.

Reference search returns nothing: check internet and try a concise health-topic term. No result means no usable source was retrieved, not that the topic does not exist.

A practical performance boundary

Saved-report access, filtering and care updates avoid model generation and should be lighter operations. Timing still depends on hardware. A smaller model may run faster but needs evaluation because it can change classification quality. Do not promise a universal response time from a single laptop trial.

13 Software tests and clinical evidence

The suite contains 30 named automated test cases. Most replace network or model calls with controlled fake results. They test software behavior, not real-world clinical performance.

What the tests exercise

The dashboard suite checks persistence, same-name encounters, sorting, assessment grouping, report access without generation, care completion, name search, returning visits and ID migrations. It tests preservation of old records and behavior when storage cannot save.

The classifier suite checks whether input reaches the selected model function, rejects unusable output, handles a narrow apostrophe issue, propagates connection failure and permits correction of malformed or inconsistent output.

Warning tests check assessment and emergency banners even when concerns are empty. They protect against the earlier regression where uncertain classification lacked an obvious warning.

Reference tests check extraction, empty results and timeout. They target _medical_search_under_test.py, a helper copy, rather than every current MedlinePlus function inside app.py. This is a test-coverage gap.

Three kinds of evidence

Evidence	What it can establish
Unit and UI tests	Known code paths behave as expected with controlled data.
Live model trials	The installed model returns outputs for selected examples on this laptop.
Clinical validation	Agreement, safety and error patterns measured against qualified clinical review. This has not been completed.

What stronger evaluation requires

Build a de-identified case set with qualified clinician labels. Include paraphrases, negation, past history, missing information, conflicting descriptions and failed generation. Record missed concerns and over-escalation. Measure latency separately from clinical agreement.

Sensitivity measures how often a system finds a condition among cases where it is present. Specificity measures how often it avoids flagging it where it is absent. Both require an agreed reference and appropriate evaluation data. We do not currently have defensible percentages for either, diagnosis accuracy, waiting-time reduction or patient outcomes.

14 Privacy and accurate project claims

Where information goes

Triage sends age, gender and the clinical description to the selected model through local Ollama. The name and reference fields are not separately placed in that request. Anything typed inside the clinical description still goes to the model, including identifying details if someone puts them there.

The database is a local file unless someone copies, shares or syncs it elsewhere. This does not automatically secure the laptop or backups. Knowledge Search sends search terms to external MedlinePlus. Do not put identifying patient details into reference queries.

Current protection limits

The prototype has no application login, role permissions, database encryption or complete audit history. File access depends on the laptop and operating system. Local storage alone does not establish compliance with health-data rules. Use fictional or de-identified data for the hackathon.

Retained reports are not an immutable medical-record system. Names and care status can change. Reopening clears the current completion timestamp. There is no separate audit table recording who changed each field and when.

Statements the team can make

We built a local prototype that uses a pretrained model to suggest provisional priorities, display uncertainty, and retain encounters for review. We added explicit returning-patient selection, care status and name/ID search. We retrieve MedlinePlus snippets for educational reference answers.

Statements the team should avoid

Do not say the team trained a clinically validated medical model, guarantees emergency detection, diagnoses conditions, performs a validated suicide-risk assessment, or safely decides who can wait. These priority tiers are prototype categories, not an implementation of a named validated scale such as ESI or Manchester Triage.

Do not claim all outputs use retrieved evidence. Triage has no guideline retrieval. Reference Search supplies snippets, but source-only instructions do not guarantee factual support for every generated sentence.

Clinical accuracy, hospital adoption, patient-outcome benefit and universal response times have not been demonstrated. These are evaluation questions, not completed features.

15 A demonstration your team can rehearse

Use fictional people and descriptions. Show the workflow rather than promising a particular label from a brief input. Live model outputs can vary.

Before presenting

Start the app, test Ollama, and confirm a demo record can be saved. Use a separate database if the main file contains material you should not display. This PowerShell example selects a separate persistent file before starting the server.

```
$env:MEDIAGENT_DB_PATH = "C:\mediagent_medical_search\data\demo.sqlite3"
python -m streamlit run app.py
```

A five minute walkthrough

Minute 1: describe the design. The browser is the interface, Python coordinates the workflow, Ollama runs Qwen, and SQLite keeps encounters. Explain that priorities require clinician review.

Minute 2: register Demo Asha as New patient with a prepared fictional description. Point to the numeric patient ID. Explain that the model gets current clinical fields while Python validates and stores its response.

Minute 3: open the report and identify priority, explanation, uncertainty and warnings. If Needs assessment appears, explain why an uncertain result stays visible instead of silently becoming low priority.

Minute 4: use Start care and Mark care completed. Search the name or ID, choose Completed patients and reopen the report. This demonstrates retention and report access without another model generation.

Minute 5: choose Returning patient and the same ID. Submit another fictional visit if time permits, then open Visit history. Show a broad MedlinePlus reference query and its source links.

Sharing the explanation

One teammate can explain the problem and dashboard. Another can explain inference and structured validation. A third can explain storage, tests and limitations. These are rehearsal roles, not a claim about who authored particular components.

If the model is slow, continue with saved demo reports and clearly identify them as saved results. If generation fails, show the retained failure record and assessment warning. Do not present an old result as a new live generation.

16 Questions judges may ask

Did your team train the model

No. We use pretrained qwen2.5:3b through Ollama. Our work includes the workflow, prompts, validation, dashboard, persistence and testing.

Why Ollama and SQLite

Ollama runs the selected model locally. SQLite keeps laptop setup simple because Python opens a database file without a separate server. This is not a claim of unlimited multi-user scale.

How does it understand new wording

The whole description goes to the language model. Python validates its response structure. This broadens the wording it can attempt to interpret, but the model can still miss or misunderstand concerns.

Is the triage decision clinically validated

No. Categories and warnings support clinician review. Software tests check application behavior. We still need clinician-labelled evaluation before making performance claims.

What happens with insufficient information

The model can return Unable to determine. Python shows an assessment warning and places it in Needs assessment. Pending and Failed classifications also remain visible.

Does completed mean the record is deleted

No. It changes care status and records a completion time. The report stays searchable. A returning visit can keep the same patient ID.

Does the AI remember previous visits

The database remembers them for the user. The classification request contains current fields, not an automatically assembled longitudinal medical history.

Is it offline and are citations guaranteed

Local triage can run after the model is downloaded. MedlinePlus search needs internet. Reference answers cite supplied source numbers, but a person must verify support.

What is the next important improvement

Controlled clinical evaluation, access control, encryption and auditing are priorities. Performance and retrieval improvements also need testing. These are future work, not already delivered features.

17 Maintaining and extending the project

Where to make a change

Begin in app.py for form labels, layout, warnings and displayed report fields. Begin in safety_classifier.py for output labels, prompt instructions, schema and parsing. Begin in patient_store.py for stored fields, searching, IDs, care status and database upgrades.

Changing the schema requires updating what the model produces, what the parser accepts, and what the UI or database expects. A field added only to a prompt can cause validation failure. A new priority label can also break sorting or warnings if other code is unchanged.

A safe edit cycle

1. Write the intended behavior in plain language, including failure behavior. Use a demo database and keep a stopped-app backup of important records.

2. Make a small coherent change. Keep test data fictional. Do not delete a database just to hide a migration error.

3. Run python -m unittest discover -v and inspect the affected interface. Add a meaningful regression test when fixing a bug.

4. Try a real model request if prompts or output changed. A mocked test alone does not show how the installed model behaves.

5. Restart Streamlit, reopen stored reports and check a return visit. Update the README and presentation when behavior changes.

Current boundaries worth improving

The dashboard counts encounters while labelling them as patients. A distinct-patient metric could reduce confusion. A central patient-profile table could make name editing consistent across visits without rewriting earlier clinical reports.

Search currently filters loaded rows in Python. More records or multiple users may require indexed SQL queries, pagination, stronger concurrency handling and a deployment design. Refresh is manual. An interrupted model request can leave a Pending row without an automatic recovery worker.

Reference tests should exercise the production parser rather than a helper copy. Retrieval could use richer approved sources and citation-support checks. These changes still require clinical and technical evaluation.

Before real clinical use, add appropriate validation, secure access, protected storage, auditing and governance. A different model is an experiment to evaluate, not an automatic quality upgrade.

18 Glossary and further reading

Term	Meaning in this project
API	A defined way to request a service. Python calls Ollama's chat API.
Inference	Generating an answer with an already trained model.
Prompt	Instructions and input sent to the model.
JSON and schema	A labelled response and rules for its allowed structure.
Token	A small piece of text the model processes or generates.
Temperature	A generation-variability setting. Zero does not ensure truth.
RAG	Retrieving source material before generating an answer.
Encounter	One recorded visit or submission. A patient can have several.
Primary key	A database identifier used to distinguish records.
Migration	An upgrade to an existing database while retaining its data.
Session state	Temporary application memory for a Streamlit session.
Mock	A controlled fake response used in a software test.

Project sources

This guide follows app.py, safety_classifier.py, patient_store.py, requirements.txt and the four test files in C:\mediagent_medical_search. README.md helps with setup, but some older paragraphs describe previous ID formats. The running code is the authority for current behavior.

Official reading

Streamlit architecture and execution
https://docs.streamlit.io/develop/concepts/architecture

Ollama chat API and structured output
https://docs.ollama.com/api/chat
https://docs.ollama.com/capabilities/structured-outputs

Python SQLite interface
https://docs.python.org/3/library/sqlite3.html

MedlinePlus Web Service
https://medlineplus.gov/about/developers/webservices/

The source files establish what our app implements. Official documentation explains the underlying technology. Neither substitutes for clinical validation.